

Feature: Calendar connection

From build-plan: feature 3

Branch: `feature/03-calendar-connection` (the workspace's `feature/NN-name` form)

Status: verified. Whole feature seen and agreed by Frank 2026-09-28; steps 3.1 to 3.4 built, checked by hand and reviewed step by step; step 3.4's review findings F-44 to F-46 fixed 2026-09-30. The checkpoint for the final review.

Approved one step at a time (`AGENTS.md` , "A spec is approved one step at a time"): Frank sees the whole feature once, then each of steps 3.1 to 3.4 gets its own yes just before it is built. Shaped by the conversation of 2026-09-28 after `/overview` , recorded below under Notes for the AI.

Goal

Let the business owner connect their own Google calendar, so the app can read when they are busy. One connection asks Google for both permissions the product needs (read busy times, and add and edit events), so nobody reconnects when bookings start being written in a later item. Prove it the way the Phase 1 exit asks: a real event in Frank's own Google calendar comes back as a busy block.

In scope

- **A person knows whose login they are.** A person (`resource`) can be linked to one login in its business. The business's first person is linked to its business owner. That link is how the app knows which person is "you". Step 3.1's blocker decides this; the rest of the spec assumes the recommended answer.
- **The `calendar_connection` table:** one per person, never one for the business. Provider, the Google account's email, the encrypted tokens, the permissions Google granted, a status, and when it was last read.
- **The token cipher:** AES-256-GCM under `CALENDAR_TOKEN_KEY` , in `packages/shared` as the plan puts "the crypto" there.
- **Connect:** the signed-in person starts it from the dashboard, Google asks for consent, the API checks that **both** permissions were granted (Google silently drops what it does not recognise, and a user can untick one) and refuses a half-connection, then stores the tokens encrypted.
- **Disconnect:** revoke at Google, then delete the connection.
- **The provider seam:** one interface the rest of the app calls ("when is this person busy?"), with Google as its first and only implementation.
- **Busy times:** a free/busy query for one person over a time range, with the access token refreshed when it has expired. A failed check never reports "free": it throws, and a refused refresh marks the

connection `needs_reconnect` .

- **A check command** that prints a person's busy blocks for the next days, used to prove the Phase 1 exit against Frank's real calendar.
- **A dashboard card** in the settings mockup's shape: not connected with a Connect button; connected, showing the Google account and when it was last read, with Disconnect; needs reconnecting, with Reconnect.

Out of scope

- **Workers connecting their own calendars.** Parked on 2026-09-28 until a client has workers (item 13 or 17). The table is one row per person from day one, so nothing is migrated then.
- **Writing bookings into Google.** The permission is asked for here; the first call that writes an event is built with the first booking (item 5). Removing an event on cancel is item 7.
- **The email that tells the business to reconnect.** Resend arrives with confirmations (item 6). Here the card shows the state; nothing is sent.
- **Background token refresh.** Tokens are refreshed when a check needs them. The job runner is item 8.
- **Microsoft, CalDAV (Apple iCloud) and ICS plugs.** Each is a new file behind the seam, built when a client needs one.
- **The Settings page.** The card sits on today's dashboard and moves into Settings in item 12.
- **Google's app verification.** Item 22, finished before item 13. Until then the app stays in Testing mode and a connection lasts seven days.
- **Railway.** No production environment variable, migration or Google client is touched. The feature is built and proved locally.

Build loop

Steps are built in order, one at a time, with `workflow.stepReview: "every"` and `workflow.checkpointCommits: "enabled"` . After the green light, nothing stops until the review (`AGENTS.md`): once a step's plan (Part 1 and Part 2) has Frank's yes, it runs straight through - build, tests and checks, tick the box, publish the page, commit and push to `feature/03-calendar-connection` , `/audit` on the step, the independent review - and stops only after the review, where its findings are talked through. The only earlier stops: the agreed plan turns out wrong, a line only Frank crosses (a package, Railway, real data, `main`), or a P0/P1 finding. Each step commit is `feat: 3.N <what it does>` . `/complete` makes the final merge on Frank's yes.

Build steps

- ☒ **3.1 A person knows whose login they are, the connection table, and the cipher.** Plan approved by Frank 2026-09-28 (Part 1 and Part 2), built the same day; every Done when check passed.

Independent review next.

- Blocker, answered by Frank 2026-09-28: **A.** How the app knows which person is you: link a person to a login, a `userId` column on `resource` (exact, and item 3b and crew sign-in need the same link). Rejected, B: take the business's oldest person (no schema change, but it breaks the day that person is renamed, removed or reordered).
- Migration 0004: `resource.userId` , `calendar_connection` , `calendar_oauth_state` (see Data / contracts). Existing businesses: their first person is linked to the member whose role is `owner` , when there is exactly one; otherwise it stays unlinked and the card says so. The first person is the one named after the business (how migration 0001 and the hook make it); with no person of that name (a renamed business) it stays unlinked too, never a guess. Changed while building: the plan said "the oldest person", but the seed makes a whole cast in one transaction, so everyone shares one `createdAt` and "oldest" was a coin toss. The name-only rule is F-35's fix, on Frank's yes, 2026-09-28.
- The `afterCreateOrganization` hook links the new first person to the business's owner, read from the membership Better Auth has just made (today the platform admin; item 3b makes it the client, with no change to this line).
- The seed links each dev business's first person to its owner account.
- `packages/shared/crypto/token-cipher.ts` with its own subpath export `./crypto` , and its tests.
- A development `CALENDAR_TOKEN_KEY` is generated into `.env` (never printed), and `.env.example` gains the three names with how to make each.
- **Done when:** `db:migrate` then `db:seed` on a fresh `scheduleads_dev` leaves `painting-dev` 's first person linked to `admin@example.com` and `clinic-dev` 's to `owner@example.com` ; the database refuses a second person linked to the same login in one business and a place linked to a login; the cipher tests pass (round trip, a tampered value refused, the wrong key refused, a key of the wrong length refused at start); the shared and backend tests and both builds pass.
- ☒ **3.2 Connect a Google calendar.** Built 2026-09-28, Frank's plan yes the same day. Differences from the plan: `google-oauth.ts` became `google-oauth-client.ts` (one export, an object, per the one-export-per-file rule); `finish-google-connect.ts` (the callback's logic) and `oauth-state-fingerprint.ts` added; `connectedAt` dropped from `/calendar/connection` (a reconnect keeps the row, so no honest value); "a plan without booking" proved with an unrecognised plan, as in 2.4. By hand: Frank connected `frankdmosquera@gmail.com` for `admin@example.com` after fixing the client's redirect URI (the old `/api/calendar/google/callback` path); the by-hand Cancel was not done (he went straight to Allow; "denied" is proved by a saved test and can be seen by hand once 3.4 adds Disconnect).
- Blocker, answered by Frank 2026-09-28: **plain `fetch` , no package.** Four documented calls (build the consent URL, swap the code, refresh, revoke), and PKCE and the state are ours either way. `google-auth-library` is Google's standard client and was passed over on purpose: its main job, holding tokens in memory and refreshing them, does not fit tokens stored encrypted per person, and free/busy is plain `fetch` either way. The price, said out loud: we write the four calls (roughly

80 lines, each with saved tests), we read Google's error answers ourselves (`invalid_grant` means reconnect), and a changed Google URL is ours to fix.

- Done by Frank 2026-09-28: an OAuth client in Google Cloud (Testing mode, Frank's Gmail as a test user, redirect `http://localhost:3001/calendar/callback`), its two values in the root `.env` (checked set, not printed). An existing client was reused with a new secret; the first real connect proves the Google-side settings.
- `POST /calendar/connect` : for the signed-in person, a single-use state and a PKCE verifier are saved for ten minutes, and the Google consent URL is returned. Planned in detail 2026-09-28 (being gone through with Frank): dashboard CORS and no-store, sign-in, a known plan and the booking module; the signed-in person is the resource in the active business whose `userId` is the session user, none is a plain refusal; the ticket row keeps the SHA-256 of the state (the value lives only in the URL) and the PKCE verifier, and making one deletes that user's expired tickets; scopes `openid email https://www.googleapis.com/auth/calendar.events.freebusy https://www.googleapis.com/auth/calendar.events.owned` (events.owned is the narrowest write scope: events on calendars the person owns; checked against Google's scope list 2026-09-28); the API refuses to start without `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET`. Files: `backend/routes/calendar-routes.ts`, `backend/lib/calendar/google-oauth.ts`, `find-signed-in-person.ts`, `create-oauth-ticket.ts`; `app.ts` and `server.ts` changed.
- `GET /calendar/callback` : the state is used up in one statement, must be unexpired and belong to the signed-in login; the code is swapped; both permissions must be present or the tokens are revoked and nothing is saved; the tokens are stored encrypted. Always ends in a redirect to the dashboard with an outcome, never a JSON page. Planned in detail 2026-09-28: five outcomes to the fixed `APP_ORIGIN` address, `connected`, `denied` (Google's `error=access_denied`), `expired` (no, stale, used or other login's ticket, or no session), `missing_permission` (revoked), `failed` (swap error, or no refresh token: revoked). The ticket is used up with one `DELETE ... WHERE id = sha256(state) AND userId = session AND expiresAt > now() RETURNING`. The Google email comes from the `id_token` returned directly by the token endpoint over TLS (no signature check needed there, OpenID Connect Core 3.1.3.7), `email_verified` required. A reconnect upserts on `resourceId`. The cipher gains an associated-data argument, the connection's `resourceId`, so a value moved to another row fails (the 3.1 reviewer's note). Files: `redeem-oauth-ticket.ts`, `save-calendar-connection.ts` new; `calendar-routes.ts`, `google-oauth.ts`, `token-cipher.ts` and its test changed.
- `GET /calendar/connection` : the signed-in person and their connection, if any, without tokens. Planned 2026-09-28 (pieces 3 and 4 agreed by Frank, and "Google first, other plugs once it works" confirmed): same middleware as connect; answers `{ person: { id, name } | null, connection: { provider, accountEmail, status, lastCheckedAt, connectedAt } | null }`, never `credentials` or `grantedScopes` (a saved test checks); a login with no person gets `person: null`, not an error; called through `dashboardApiClient`.
- The dashboard card: Connect, the outcome notice after Google sends the browser back, and the connected line. Planned 2026-09-28 (piece 5 agreed): on the home under the booking links until feature 12; `frontend/components/calendar/calendar-connection-card.tsx`; Connect posts

through `dashboardApiClient` and assigns `window.location`; the `?calendar=` outcome is shown in plain words and removed with `history.replaceState`; `person: null` shows a line and no button; a one-line loading state only (3.4 adds needs reconnecting, Disconnect and API down).

- Part 1 agreed by Frank 2026-09-28 (piece 6 included). Part 2 rewritten the same day to match the pieces, being gone through: the saved tests below plus the ticket fingerprint and cleanup, `denied` and `failed` outcomes, no refresh token revoked, reconnect keeps one row, the fixed redirect, `/calendar/connection` without tokens or scopes, `person: null`, the lock refusing a value moved to another person, planted faults (ticket not used up, scope check removed, lock not bound); by hand also Cancel once and the API refusing to start without the Google values.
- **Done when:** route tests against the local database, with Google's endpoints faked, prove: no session is 401; a plan without booking is 403; a login with no person gets a clear refusal; an unknown, expired, reused or other login's state ends in `expired` and saves nothing; a consent missing either permission ends in `missing_permission`, calls revoke and saves nothing; a full consent saves one row whose stored credentials do not contain the refresh token in plain text. By hand: Frank, signed in as `admin@example.com`, connects his real Google and the card shows his Google address.
- ☒ **3.3 The provider seam and busy times.** Planned with Frank 2026-09-29 (pieces 1 to 4) and 2026-09-30 (Part 2, two tests added), built 2026-09-30. Agreed in the plan: the refresh call lives in `google-oauth-client.ts` (it already owns the token address) and the plug reuses it; a refused refresh is its own `CalendarReconnectNeededError`; a key with under a minute left is refreshed; a connection already `needs_reconnect` throws without calling Google; the fresh key (and the `needs_reconnect` mark) is written only if the row is unchanged since it was read, and a changed row is read once more; `calendar:check` defaults to 7 days, takes the time zone from the business's hours and refuses a business without them; the local `*_dev` guard moved into one shared function, `packages/shared/helpers/assert-local-dev-database.ts` (export `./assert-local-dev-database`), used by the seed, the three database test files and the command. Differences from the plan, added while building: `calendar-reconnect-needed-error.ts` (one export per file); `safe-error-reason.ts`, moved out of `warn-connect-failed.ts` so the command prints a database error by its code only; `backend/scripts/find-calendar-check-target.ts` and its test, so the refusals are testable; days capped at 31; a Google refresh that fails for any reason but `invalid_grant` throws and leaves the connection `connected` (Google down must not disconnect anyone); the busy blocks are read from the one calendar Google returns, whatever key it uses. By hand: Frank's event "tomorrow 8:00 to 9:00" (not 10 to 11, same proof) printed as `Thu 1 Oct, 08:00 to 09:00` in America/Edmonton.
- Blocker: **which of his Google calendars count as busy. Answered A, Frank, 2026-09-29: the main calendar only**, no new permission. Every calendar he owns would need `calendar.calendarlist.readonly` and a filter for subscribed ones (a holiday calendar would block whole days). Which calendars count is the owner's choice, so the picker is feature 12's; nobody real connects before feature 13, so adding that permission then costs no client a reconnect.

- `backend/lib/calendar/calendar-provider.ts` : the seam's type. `google-calendar-provider.ts` : free/busy, refresh, revoke. `get-busy-times.ts` : loads the person's connection, refreshes when needed, saves the new access token encrypted, records `lastCheckedAt` , and returns the busy blocks.
- A person with no connection has no Google busy times: an empty list, not an error. A connection that fails is never an empty list.
- `npm run calendar:check --workspace=backend -- <login email> [days]` prints that person's busy blocks in the business's time zone. Refuses any database that is not local and `*_dev` , like the seed.
- **Done when:** tests with Google faked prove: an expired access token is refreshed once and the new one saved; Google refusing the refresh marks the connection `needs_reconnect` and throws; a Google error, a time-out or a per-calendar error throws; no connection returns an empty list. Added 2026-09-30 with Frank: a key with time left is not refreshed; a connection already `needs_reconnect` throws without calling Google; a reconnect at the same moment is not overwritten; `calendar:check` refuses a business with no hours; the shared guard has its own test (the Railway tunnel refused). Planted faults: a failure answering "free", the same-moment guard removed. By hand (the Phase 1 exit): an event Frank creates tomorrow 10:00 to 11:00 in his Google calendar is printed by `calendar:check` as a busy block at those times.
- ☒ **3.4 Disconnect, and the card's other states.** Planned with Frank and built 2026-09-30. Agreed in the plan: the permission is handed back through the seam (`findCalendarProvider` , moved out of `get-busy-times.ts`), then the row is deleted whatever the provider said; **the shared-Gmail rule** (Frank's yes, Sep 30): Google keeps one permission per Google account for our app, so it is handed back only when no other connection (in any business) uses the same Gmail, in Disconnect and in F-37 alike (`hand-back-calendar-permission.ts`); keys that cannot be opened are still deleted; the answer names one of four things (contract below); a connection already `needs_reconnect` says Google had already stopped accepting it; the card shows "Last read ... ago", gets Disconnect in both connected states (Reconnect too when it needs it), no confirm box, Try again when the API does not answer; F-37 compares Gmail addresses, not Google's account ID (rare address change heals on the next reconnect); F-40's list is `packages/shared/calendar/calendar-connect-outcomes.ts` (export `./calendar`) with `isCalendarConnectOutcome` , tested in shared, so the frontend needs no test tool. Differences found while building: the answer's field is `atProvider` (Disconnect goes through the seam), not `atGoogle` ; with the whole API down the dashboard shows its own "Cannot reach the API" page first, so the card's own line only appears when the calendar call alone fails (proved by blocking that one call in the browser). By hand: Frank's Disconnect emptied the card and Google's apps page no longer listed the app; his Reconnect came back connected, a new row.
 - `POST /calendar/disconnect` : revokes at Google, then deletes the row. If Google does not answer, the row is still deleted and the answer says so, so the card can tell him to remove access in his Google account.
 - The card: needs reconnecting (with Reconnect, the same connect flow), loading, and the API not answering.

- Carried from 3.2's review, on Frank's call: **F-37**, a reconnect with a different Google account hands the old account's permission back to Google (best effort, after the new row is saved; the same account is left alone); **F-40**, the card's outcome words are typed to the API's five outcomes and looked up only as the card's own keys, so a made-up `?calendar=` shows nothing and a renamed outcome fails the build.
- **Done when:** tests prove disconnect calls revoke and deletes the row, still deletes it when revoke fails and reports that, and refuses a login with no person. By hand: Disconnect empties the card, and Google's "Your connections to third-party apps" page no longer lists the app; a connection set to `needs_reconnect` in the database shows Reconnect. F-37: a test reconnects with another Gmail and sees the old refresh token handed back, and the same Gmail not. F-40: a renamed outcome fails the frontend build; `?calendar=constructor` shows no notice. Added 2026-09-30 with Frank: Google silent or keys unreadable still deletes; the shared-Gmail rule in Disconnect and F-37; `already_stopped`; only your own connection is deleted, a coworker's stays; 404, 401 and 403; F-37 with Google silent still ends connected with one log line; planted faults (delete without handing back, the shared-Gmail check removed, the same-Gmail check removed, an outcome renamed in the list and the API but not the card); by hand also the API down with Try again, and "Last read ... ago".

Files / areas

- `packages/shared/db/booking-tables/resource-table.ts` (changed: `userId`)
- `packages/shared/db/booking-tables/calendar-connection-table.ts` (new)
- `packages/shared/db/booking-tables/calendar-oauth-state-table.ts` (new)
- `packages/shared/db/index.ts` (changed)
- `packages/shared/migrations/0004_*.sql` (new, generated, then checked)
- `packages/shared/crypto/token-cipher.ts`, `token-cipher.test.ts` (new)
- `packages/shared/package.json`, `tsconfig.json`, `tsconfig.build.json` (changed: the `crypto` folder and its export)
- `packages/shared/scripts/seed-dev.ts` (changed: link first people)
- `backend/lib/auth/auth-server.ts` (changed: the hook links the creator)
- `backend/lib/calendar/` (new): 3.2 `google-oauth-client.ts`, `find-signed-in-person.ts`, `create-oauth-ticket.ts`, `redeem-oauth-ticket.ts`, `oauth-state-fingerprint.ts`, `save-calendar-connection.ts`, `finish-google-connect.ts`, `warn-connect-failed.ts` (+ test, F-38); 3.3 `calendar-provider.ts`, `google-calendar-provider.ts`, `get-busy-times.ts` (+ test), `calendar-reconnect-needed-error.ts`, `safe-error-reason.ts`; `google-oauth-client.ts` (changed: refresh)
- 3.3 `packages/shared/helpers/assert-local-dev-database.ts` (+ test, new), `packages/shared/scripts/seed-dev.ts` and the two route test files (changed: use it), `backend/scripts/find-calendar-check-target.ts` (+ test, new)

- 3.4 `backend/lib/calendar/find-calendar-provider.ts` , `hand-back-calendar-permission.ts` , `disconnect-calendar.ts` (new); `finish-google-connect.ts` , `get-busy-times.ts` , `calendar-routes.ts` (+ test), `refuse.ts` (changed); `packages/shared/calendar/calendar-connect-outcomes.ts` (+ test, new) and the shared `package.json` and both tsconfigs (changed: the `calendar` folder and its export); `frontend/components/calendar/calendar-connection-card.tsx` and `frontend/lib/api-client.ts` (changed)
- `backend/routes/calendar-routes.ts` and `calendar-routes.test.ts` (new)
- `backend/app.ts` (changed: mounts `/calendar` , CORS and no-store), `backend/server.ts` (changed: refuses to start without the Google values)
- `backend/lib/errors/refuse.ts` (changed: one new code, `no_person`)
- `backend/scripts/calendar-check.ts` (new), `backend/package.json` (changed: `calendar:check`)
- `frontend/components/calendar/calendar-connection-card.tsx` (new)
- `frontend/lib/api-client.ts` , `frontend/app/page.tsx` (changed)
- `.env.example` (changed), `AGENTS.md` Commands (changed: `calendar:check`)

Data / contracts

`resource.userId`

- `text` , nullable, FK `user.id` on delete set null .
- Unique per business: a unique index on (`organizationId` , `userId`) where `userId` is not null. One login is at most one person in a business.
- A check: `userId` is null, or `kind` is `person` . A place is never a login.
- Never set from a request in this feature. Set by the migration, the hook and the seed.

`calendar_connection`

- `id` text, `randomUUID()` .
- `organizationId` text not null, FK `organization.id` on delete cascade .
- `resourceId` text not null; composite FK (`organizationId` , `resourceId`) to `resource` (`organizationId` , `id`) on delete cascade , the same pattern as `availability_rule` , so a connection can only name a person of its own business. Unique on `resourceId` : one connection per person.
- `provider` text not null, check in (`google`).
- `accountEmail` text not null: the Google account's address, from the `id_token` Google returns with the tokens. Shown on the card.
- `credentials` text not null: the cipher's output (below) of the JSON `{ "refreshToken": string, "accessToken": string, "accessTokenExpiresAt": ISO-8601 string }` . Never returned by any

route, never logged.

- `grantedScopes` text not null: the space-separated list Google granted.
- `status` text not null default `connected`, check in (`connected`, `needs_reconnect`). Disconnecting deletes the row, so there is no `disconnected` state holding dead tokens.
- `lastCheckedAt` timestampz, nullable: the last successful busy read.
- `createdAt`, `updatedAt` timestampz, as every built table.

calendar_oauth_state

- `id` text: SHA-256 (hex) of the state value. The value itself (32 random bytes, base64url) exists only in the consent URL.
- `userId` FK `user.id` cascade, `organizationId` FK cascade, `resourceId` (composite FK as above).
- `codeVerifier` text: the PKCE verifier (43 base64url characters).
- `expiresAt` timestampz: ten minutes after it is made.
- Used up by one `DELETE ... WHERE id = sha256(state) AND userId = <session's user> AND expiresAt > now() RETURNING ...`, so two callbacks with the same state cannot both succeed, and another login's or a stale ticket is never touched. Expired rows are deleted when a new state is made for the same login.

The cipher

- `encryptCredentials(plaintext: string, key: Buffer, boundTo: string): string` and `decryptCredentials(value: string, key: Buffer, boundTo: string): string`. `boundTo` is the connection's `resourceId`, sealed in as GCM additional data and not stored: a value only opens for the person it was locked for, so one copied onto another person's row is refused.
- Output: `v1.<iv>.<ciphertext>.<tag>`, each base64url; a fresh 12-byte IV per call; 16-byte tag. `v1` lets the key or format change later without guessing.
- `readTokenKey(): CALENDAR_TOKEN_KEY` is 32 bytes, base64. Missing or the wrong length stops the API at start, like `BETTER_AUTH_SECRET`.
- A tampered value, a wrong key, another person or an unknown version throws; never returns garbage.

Google

- Consent URL: `https://accounts.google.com/o/oauth2/v2/auth` with `response_type=code`, `access_type=offline`, `prompt=consent` (so a refresh token comes back every time), `code_challenge_method=S256`, the state, and the redirect `${BETTER_AUTH_URL}/calendar/callback`. `BETTER_AUTH_URL` is already the API's own origin, so no new variable.
- Scopes: `openid`, `email`, `https://www.googleapis.com/auth/calendar.events.freebusy` (read busy times; not `calendar.freebusy`, the first repo's trap), and

`https://www.googleapis.com/auth/calendar.events.owned` (add and change events on calendars the person owns), checked against Google's scope list on Sep 28.

- Required to count as connected: both calendar scopes present in the granted list. `openid email` alone, or one calendar scope, is a half-connection.
- Token swap and refresh: `https://oauth2.googleapis.com/token` . A refresh answered with `invalid_grant` means Google no longer accepts the saved permission (Testing mode's seven days, or access removed): `needs_reconnect` .
- Revoke: `https://oauth2.googleapis.com/revoke` .
- Free/busy: POST `https://www.googleapis.com/calendar/v3/freeBusy` with `items: [{ id: "primary" }]` : the main calendar only (3.3's blocker, answered A).

Routes

All under `/calendar` , behind `dashboardCorsMiddleware` and `dashboardNoStoreMiddleware` . All but the callback also run `requireOrganizationMiddleware` , `requireKnownSubscriptionMiddleware` and `requireModuleMiddleware("booking")` . The business and the person always come from the session, never from the request. "The signed-in person" is the person in the active business whose `userId` is the session's user.

```
| Route | Answer | |---|---| | GET /calendar/connection | 200 { person: { id, name } \ | null,
connection: { provider, accountEmail, status, lastCheckedAt } \ | null } || POST
/calendar/connect | 200 { url } ; 409 no_person when the login has no person here || GET
/calendar/callback | always 302 to ${APP_ORIGIN}/?calendar=<outcome> || POST
/calendar/disconnect | 200 { atProvider: "handed_back" \ | "not_confirmed" \ |
"still_used" \ | "already_stopped" } ; 404 not_found when there is nothing to disconnect; 409
no_person |
```

`atProvider: handed_back` the provider confirmed; `not_confirmed` it did not answer, or our keys could not be opened; `still_used` another connection uses the same account, so the permission stays; `already_stopped` the connection was `needs_reconnect` and the hand-back was not confirmed. The row is deleted in every case.

Callback outcomes, the only values `?calendar=` ever carries (written once in `packages/shared/calendar/calendar-connect-outcomes.ts`): `connected` , `denied` (the person pressed Cancel at Google), `missing_permission` , `expired` (state unknown, expired, used, or another login's; or no session), `failed` (Google did not answer or refused the code). The redirect target is fixed; nothing from the request picks it, so there is no open redirect.

401, 403 and the refusal shape are the existing ones in `refuse.ts` .

Testing

Vitest, beside the code, run by the existing commands.

- `packages/shared` : the cipher (step 3.1).
- `backend` : the OAuth helpers and the scope check (3.2), the busy-times function with Google faked (3.3), the routes against the local seeded `scheduleads_dev` with Google faked (3.2, 3.4). Google is never called from a test; the fake answers are the documented shapes.
- Each new test is proved able to fail once, as in feature 2.
- By hand, never in a test: the real consent screen, the real busy block, Google's third-party apps page.

Notes for the AI

- **Agreed 2026-09-28, before this spec.** The app holds the only real copy of every booking; Google is an optional side pipe per person (their busy times in, a copy of their bookings out, nothing read back), so a worker deleting a copy loses nothing. The scheduler (items 12b, 19) is our own and needs no Google. Feature 3 connects the owner only; the table is still one per person. Other providers are plugs behind the seam, built when a client needs one; at onboarding the question is "iPhone or Android?".
- **Written fresh.** The plan speaks of porting the first repo's calendar code. That code is not in the workspace, so this feature writes it from the design and the traps the plans record. Nothing depends on the old code.
- **Traps carried from the plans.** Google silently drops an unknown scope and consents to the rest, so the granted list is always checked. Losing `CALENDAR_TOKEN_KEY` makes every stored connection unreadable; the dev key lives only in `.env`, and the production key is Frank's to set on Railway at deploy.
- **Tokens never leave the backend.** No route returns them, no log prints them, the check command prints only busy blocks.
- **Do not decide for the owner** (decision 30): the product offers the connection; nothing connects or reads by default.
- The permission check rule stands: code asks what a login may do, never `role === "owner"`. This feature needs no role check: a person may only connect or disconnect their own calendar, which the `userId` link decides.
- The card follows `prototypes/settings.html` (`conn--ok` , `conn--bad`) with the dashboard's existing components.

Open questions

Each is a blocker on its own step, answered when that step is planned:

1. Step 3.1: how the app knows which person is you. **Answered A, Frank, 2026-09-28:** link a person to a login. 2. Step 3.2: plain `fetch` or `google-auth-library` (recommended: `fetch`). 3. Step 3.3: which of his Google calendars count as busy. **Answered A, Frank, 2026-09-29:** the main one only; the owner's picker is feature 12.

Implementation walkthrough

What was actually built, by area. The steps above are the plan; the build log records how each step went against it. This is the shape of the code the feature left behind, and the decisions in it that are not visible from the code alone.

packages/shared

- `db/booking-tables/resource-table.ts` gained `userId` : a person can be linked to one login in its business (a partial unique index on `(organizationId, userId)` , and a check that only a `person` can be linked). That link is how the app knows which person is "you".
- `db/booking-tables/calendar-connection-table.ts` : one row per person, never one for the business. Provider, the Google address shown on the card, the locked credentials, the granted scopes, `status` (`connected` or `needs_reconnect`) and `lastCheckedAt` . A composite foreign key keeps a connection from naming another business's person. `calendar-oauth-state-table.ts` holds the ten-minute connect tickets by the SHA-256 of their state, never the state itself.
- `migrations/0004_calendar_connection.sql` adds both tables and `userId` , and links each existing business's first person (by the business's name) to its owner; a renamed business stays unlinked rather than guessed (F-35).
- `crypto/token-cipher.ts` : AES-256-GCM under `CALENDAR_TOKEN_KEY` , a versioned `v1.` value, a fresh IV per value, and the person's id sealed in as associated data, so a locked value copied onto another row does not open.
- `calendar/calendar-connect-outcomes.ts` is the one list of the five connect outcomes; the API's return address and the dashboard card are both typed by it, and `isCalendarConnectOutcome` reads only those five from the address bar.
- `helpers/assert-local-dev-database.ts` is the one "local `*_dev` database only" guard, used by the seed, the three database test files and `calendar:check` (it replaced two copies).
- `scripts/seed-dev.ts` links each dev business's first person to its owner.

backend

- `lib/calendar/google-oauth-client.ts` is the whole conversation with Google's sign-in service in plain `fetch` (no Google package, Frank's call in 3.2): the consent address with PKCE and both narrowest calendar scopes, the code swap, the refresh (`invalid_grant` becomes `CalendarReconnectNeededError`), reading the Gmail from the sign-in token (issuer, audience and `email_verified` checked), and revoke. Every call has a ten-second limit.
- The connect flow: `create-oauth-ticket.ts` makes the ticket, `redeem-oauth-ticket.ts` uses it up in one `DELETE ... RETURNING` bound to the session's login and expiry, and `finish-google-connect.ts` turns everything Google sends back into one of the five outcomes. It saves only when both permissions, a refresh token and a readable identity are present; when it gives up it hands the new tokens back unless that Gmail has a working connection anywhere, the person's own included (F-

44, F-47). A reconnect with a different Gmail hands the old one back after the new one is saved (F-37). `warn-connect-failed.ts` leaves one log line per failure, through `safe-error-reason.ts`, which reduces a database error to its code.

- The seam: `calendar-provider.ts` is the shape every calendar plug follows (busy blocks, a fresh key, revoke); `google-calendar-provider.ts` is the Google plug (free/busy for `primary` only, read as the one calendar Google returns); `find-calendar-provider.ts` picks the plug by the connection's provider.
- `get-busy-times.ts` answers "when is this person busy?": no connection is an empty list, a `needs_reconnect` one throws, a key with under a minute left is refreshed and saved locked only onto the row as it was read (read once more if it changed), and any failure throws, never "free".
- Disconnect: `disconnect-calendar.ts` hands the permission back through `hand-back-calendar-permission.ts`, then deletes the row whatever the provider said, answering one of `handed_back`, `not_confirmed`, `still_used`, `already_stopped`. `is-calendar-account-in-use.ts` is the shared-Gmail rule: Google keeps one permission per Gmail for the app, so it is never handed back while a working connection uses it.
- `routes/calendar-routes.ts`: `GET /connection`, `POST /connect`, `GET /callback` (always a 302 to the fixed dashboard address) and `POST /disconnect`, all with the business and person from the session. `app.ts` mounts CORS, Hono's CSRF origin check (`dashboard-csrf-middleware.ts`, F-46) and no-store on `/calendar/*`; `server.ts` refuses to start without the Google values.
- `scripts/calendar-check.ts` (`npm run calendar:check`) prints a login's busy blocks in the business's time zone; `find-calendar-check-target.ts` finds whose calendar, and refuses an unknown login, no person, two businesses and a business with no hours.

frontend

- `components/calendar/calendar-connection-card.tsx` on the dashboard home: Connect, the outcome line after Google, "Last read ... ago", Disconnect with one line saying what happened at Google, the red "needs reconnecting" row with Reconnect and Disconnect, and Try again when the calendar call fails. It moves into Settings in feature 12.
- `lib/api-client.ts` gained the three calendar calls through the typed `dashboardApiClient`.

Decisions worth knowing later

- Google is an optional side pipe per person: busy times in, later a copy of bookings out. The app holds the only real copy of every booking.
- Only the main calendar is read (Frank, Sep 29); which calendars count is the owner's picker in feature 12, and adding `calendar.calendarlist.readonly` then costs no client a reconnect.
- Accounts are told apart by Gmail address, not Google's account ID; a rare address change heals on the next reconnect.
- With the whole API down, the dashboard's own "Cannot reach the API" page shows before the card; the card's own line appears only when the calendar call alone fails.

- Production cookies and the CSRF check assume the dashboard and the API share one site (`app.` and `api.` of one domain).

Carried forward, not done here

- Workers connecting their own calendars: parked until a client has workers (item 13 or 17). The table is already one row per person.
- Writing bookings into Google (item 5) and removing them on cancel (item 7); the email telling a business to reconnect arrives with Resend.
- `CALENDAR_TOKEN_KEY` and the Google values on Railway are Frank's to set at deploy; losing the key makes every stored connection unreadable.
- Two rare races stay unguarded, both harmless in effect (the final review's remaining risk): two connects of one person with different Gmails at the same moment, and a Disconnect finishing just as a Reconnect saves.

Findings

Resolved during this feature and archived with it. IDs carry the feature number; the bare IDs inside each entry are the ones used while the work was live.

3/F-35 [P3] closed - On a renamed seeded business the backfill links a worker to the owner, and the seed then crashes

File: packages/shared/migrations/0004_calendar_connection.sql:51 **Found:** 2026-09-28 by /audit independent (scope: step 3.1; lens: all) **Why it matters:** When no person carries the business's current name, the backfill falls back to the oldest person, and on seed-made data every person shares one `createdAt`, so the pick is the lowest random id. Reproduced on a throwaway database at 0003 holding a seed-shaped `painting-dev` whose business had been renamed ("Summit Painting"): migration 0004 linked `admin@example.com` to "Marco (estimator)", and the next `db:seed` then aborted with `resource_organization_user_unique`, because its link at `scripts/seed-dev.ts:330` targets the person named after the business while the login is already on another person in that business. An owner can rename a business today (the `owner` role has `organization: ["update"]`), though no screen offers it. Production is not affected in practice: a real first person was made by 0001 or the hook, before anyone else, so "oldest" is right there. All other edge cases behaved: two owners and a business with only places stay unlinked, the person beats an older place of the same name, the named person beats an older one, and a renamed business with real ages links the oldest. **Suggested fix:** Either link in the migration only when a person carries the business's name (leave the rest unlinked, as for two owners, which is what the comment's "rather than guess" promises), or have the seed skip the link with a printed note when the login is already on another person in that business. **Resolution:** Fixed 2026-09-28 on Frank's yes, the first option: migration 0004 links only a person named after the business, oldest breaking a tie between two of that name; no such person leaves the business unlinked. 0004 had only run on the local dev database

(with the right result), so it was corrected before reaching Railway. Proved on a throwaway database at 0003 with a renamed seed-shaped business: the fixed 0004 left it unlinked, linked the clinic's named person over a same-instant colleague, and the next db:seed linked the named person and finished; the committed 0004 (9eba7b5) on the same data linked "Marco (estimator)". Throwaway database dropped. Closed 2026-09-28 by /audit independent (scope: step 3.2, re-examining 7c6b593): the backfill at lines 38-55 now picks only a person whose name equals the business's name, oldest then id breaking a tie, and joins it to a business with exactly one owner; a renamed business matches no row and stays unlinked, so the guess is gone and the seed's link to the named person can no longer collide. The repair adds nothing new: the unique index and the person-only check above it are unchanged, and the shared and backend tests pass.

3/F-36 [P3] closed - Two comments in this step break the comment standard

File: .env.example:63 **Found:** 2026-09-28 by /audit independent (scope: step 3.1; lens: quality) **Why it matters:** `coding-standards.md` (Comments, "The balance") keeps history out of code comments and most comments to one or two lines. This step rewrote the `WIDGET_ORIGINS` note to "Read by the public routes since step 2.4" (history, a step number), and grew the `afterCreateOrganization` comment (`backend/lib/auth/auth-server.ts:105-109`) to five lines covering three separate points. Harmless at runtime; it is the pattern the next edit copies, as F-34 found for the tsconfigs. **Suggested fix:** "Read by the public booking routes." for the first. For the hook, keep two lines beside the code: why the person is named after the business and linked to the owner, and why it sits outside the create transaction; the note about item 3b fits in one short clause or the build log. **Resolution:** Fixed 2026-09-28, half by change and half by clarifying the rule with Frank. The `.env.example` note now reads "Read by the public booking routes." with no step number. The hook comment stays: Frank clarified that the comment rule is a guide, not a line count (comments only where they help, beside their line, as long as needed), and those lines each explain a why at the line they describe. `coding-standards.md` "The balance" now says so. Closed 2026-09-28 by /audit independent (scope: step 3.2): `.env.example:63` reads "Read by the public booking routes." with no step number, and the step 3.2 edits to the same file carry none either. The hook comment in `auth-server.ts` is unchanged and, under the clarified rule ("a guide, not a line count"), each of its lines explains a why beside the code it describes, so it no longer breaks the standard. Kept on Frank's call, not the reviewer's.

3/F-37 [P3] closed - Switching to another Google account leaves the old account's permission live at Google

File: `backend/lib/calendar/save-calendar-connection.ts:44` **Found:** 2026-09-28 by /audit independent (scope: step 3.2; lens: security) **Why it matters:** When a person connects again, the new tokens simply overwrite the old ones. If the second connect is a different Google account (the saved test "a reconnect replaces the row" does exactly that), the first account's permission is never handed back: Google still lists the app under that account's third-party access, and we no longer hold the token that could remove it, so step 3.4's Disconnect cannot either. Nothing leaks (we threw the old token away), but the owner is left with a permission they cannot see from the dashboard. Today the card only offers Connect when there is

no connection, so this is reached by the API or by 3.4's Reconnect. **Suggested fix:** In the save (or in `finish-google-connect.ts` just before it), read the existing row; when its account email differs from the new one, open the old credentials and hand the old refresh token back, best effort, after the new row is saved. Leave the same-account case alone: that is one permission at Google, and revoking it would kill the new tokens too. **Resolution:** Carried to step 3.4 on Frank's call, 2026-09-29: it belongs with Disconnect and Reconnect, which 3.4 builds. Written into 3.4's plan and Done when in the spec. Repaired by step 3.4 (533b662), found so by /audit independent (scope: step 3.4), 2026-09-30, and set to `fixed` by that pass (the builder had left it `open`): `finish-google-connect.ts:82-95` reads the person's existing row before the save, and after a successful save (`:117-127`) a different Gmail (compared without case) hands the old refresh token back through `handBackCalendarPermission` , best effort, skipped when another connection still uses that Gmail, with one log line and no token when Google does not confirm. The same Gmail is left alone. Four route tests (`calendar-routes.test.ts:462-500`) cover the hand-back after the save, the same Gmail in another case, a Gmail still used elsewhere, and Google silent. The repair introduces nothing new of its own; the other hand-back paths in the same file are F-44. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: current, feature 3's final review at d76d355): re-read `finish-google-connect.ts:94-140` . The old row is read before the save, the hand-back runs only after a successful save and only for a different Gmail (compared without case), goes through `handBackCalendarPermission` with the person's own row left out (which by then holds the new Gmail, so only other connections count), and its failures are caught into one log line with no token. Route tests `calendar-routes.test.ts:465-505` pass. Nothing new introduced.

3/F-38 [P3] closed - When Google refuses a connect, nobody can see why

File: backend/lib/calendar/finish-google-connect.ts:55 **Found:** 2026-09-28 by /audit independent (scope: step 3.2; lens: quality) **Why it matters:** Every way the trip back from Google can go wrong ends in the same "failed" and nothing is written anywhere. The code swap's error (`Google refused the code swap (401)` , a wrong secret or redirect) is thrown away at line 55, a failed save at line 83, and anything unexpected at `calendar-routes.ts:89` . The owner sees "try again in a minute", and the person fixing it has no clue whether it was the client secret, the redirect address, the token key or the database. The step's own by-hand connect needed a redirect fix; the next one on Railway will have only this. **Suggested fix:** One `console.warn` at each of those three catches with the outcome and Google's HTTP status or error code (`invalid_client` , `invalid_grant`), never a token or the code itself. **Resolution:** Fixed 2026-09-29 on Frank's yes. Every "failed" path of the callback now leaves one line, `[calendar] connect failed at <step>: <reason>` , through `backend/lib/calendar/warn-connect-failed.ts` : Google's error other than Cancel, no code, the code swap (now carrying Google's own code word, e.g. `400 invalid_grant` , never its free text), the token check (no refresh token, or a sign-in token refused), the save, and the route's last catch. A database error is logged as its Postgres code only, because its message carries the query and its values. Denied, expired and missing permission are the person's own doing and log nothing. Tests: each failed route test checks its exact line, the full consent checks there is none, every test checks no token or locked value reaches a warning, and a unit test checks a failed query logs only `database error 23505` . Five faults planted (raw message logged, swap line removed, Google's word dropped, save line removed, a token put in a line), each caught. 63 backend tests pass. Waits for the next

review to close. Closed 2026-09-30 by /audit independent (scope: step 3.3, re-examining f9601b3): every failed path in `finish-google-connect.ts` (Google error, no code, the code swap, the token check, the save) and the route's last catch (`calendar-routes.ts:91`) still leaves one `[calendar] connect failed at ...` line. Step 3.3 moved the reason logic into `safe-error-reason.ts` unchanged (a failed query still logs only its Postgres code), and `warn-connect-failed.test.ts` plus the route tests that check each exact line and that no secret reaches a warning still pass. Nothing new introduced.

3/F-39 [P3] closed - The Google identity check and three give-up paths have no saved test

File: backend/lib/calendar/google-oauth-client.ts:110 **Found:** 2026-09-28 by /audit independent (scope: step 3.2; lens: tests) **Why it matters:** `readIdentity` decides which Google address is saved and shown on the card, and refuses a sign-in token meant for another app, from another issuer or with an unverified address. No test sends any of those, so removing the audience or `email_verified` check would pass the whole suite. The same holds for three other paths in `finish-google-connect.ts`: the person unlinked from the login during the ten minutes at Google (line 51, should be `expired`), a Google error other than Cancel (line 35, `failed`), and a save that fails (line 83, tokens handed back). Separately, the "connection comes without the tokens" test (`calendar-routes.test.ts:383`) only passes because the reconnect test above it ran first, so running it alone fails. **Suggested fix:** Add route tests with the fake token answer carrying a wrong `aud`, a wrong `iss` and `email_verified: false` (each ends in `failed`, hands the token back, saves nothing), plus one each for the unlinked person and `error=server_error`. Have the connection test make its own connection first. **Resolution:** Fixed 2026-09-28 on Frank's yes, tests only (`backend/routes/calendar-routes.test.ts`). Six new route tests: a sign-in token for another app, from another issuer, or with an unverified email each end in `failed`, hand the refresh token back and save nothing; a person unlinked during the trip ends in `expired`; `error=server_error` ends in `failed` and uses the ticket up; a save that fails (the lock given a bad key for that one call) hands the tokens back and ends in `failed`. The connection and reconnect tests now make their own connections, and every test starts with Ana unconnected (a `beforeEach` delete), so none depends on another: each passes run alone. Proved: six faults planted one at a time (audience, issuer and verified-email checks removed, the person re-check removed, every Google error treated as Cancel, no hand-back after a failed save), each failing exactly its own test and nothing else; files restored and compared. 61 backend and 42 shared tests, both builds, lint and format pass. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: step 3.3): the six tests are in `calendar-routes.test.ts:358-411` and assert the outcome, the hand-back, that nothing is saved and the exact warning; the connection and reconnect tests make their own connections and a `beforeEach` clears Ana's, so none depends on another. Step 3.3 only swapped the file's database guard for the shared one. 79 backend tests pass.

3/F-40 [P3] closed - The card's list of outcomes is not tied to the API's, and a made-up one shows an empty red box

File: frontend/components/calendar/calendar-connection-card.tsx:17 **Found:** 2026-09-28 by /audit independent (scope: step 3.2; lens: quality) **Why it matters:** `OUTCOMES` is a plain `Record<string, ...>`

looked up with whatever `?calendar=` says (line 71). An inherited name such as `/?`
`calendar=constructor` finds a built-in function, so the card shows an empty error box. Harmless, but it also means the five words are typed only on the backend (`ConnectOutcomeType`): rename one there and the card silently shows nothing, and the build does not notice. The spec already disagrees on one of them (see F-41). **Suggested fix:** Type the map's keys as the five outcomes and look up with `Object.hasOwn(OUTCOMES, outcome)` ; ideally the outcome list lives once in `packages/shared` so both sides import it. **Resolution:** Carried to step 3.4 on Frank's call, 2026-09-29: 3.4 reworks the card's states anyway. Written into 3.4's plan and Done when in the spec. Repaired by step 3.4 (533b662), found so by `/audit independent (scope: step 3.4)`, 2026-09-30, and set to `fixed` by that pass (the builder had left it `open`): the five outcomes live once in `packages/shared/calendar/calendar-connect-outcomes.ts` ; the backend's `finishGoogleConnect` and the callback route are typed by it, the card's `OUTCOMES` is a `Record` keyed by it (so a rename on either side fails a build), and the address value is looked up only after `isCalendarConnectOutcome` , an equality check against the list, so `constructor` or `__proto__` shows nothing (shared test `calendar-connect-outcomes.test.ts`). Nothing new introduced. Waits for the next review to close. Closed 2026-09-30 by `/audit independent (scope: current, feature 3's final review at d76d355)`: re-read `calendar-connection-card.tsx:27-42,79-82` , `calendar-routes.ts:107` and `packages/shared/calendar/calendar-connect-outcomes.ts` . The card's map is keyed by `CalendarConnectOutcomeType` , the callback's outcome is typed by it, and the address value is used only after the equality check. The shared tests and the frontend build pass. Nothing new introduced.

3/F-41 [P3] closed - The spec's contracts still describe the shapes step 3.2 changed

File: blueprint/context/current-feature.md:374 **Found:** 2026-09-28 by `/audit independent (scope: step 3.2; lens: quality)` **Why it matters:** Steps 3.3 and 3.4 build on the Data / contracts section, and three lines there no longer match the code: the Routes table names the Cancel outcome `declined` while the API and card say `denied` (line 374); the cipher is still written with two arguments, without the person it is now sealed to (line 326), which 3.3 needs to open the tokens; and the ticket is said to be used up by id alone (line 320), while the code (rightly) also requires the same login and an unexpired ticket. The project's rule is that a wrong spec is corrected before the next step builds on it. **Suggested fix:** Update those three lines to match the code before 3.3's plan is written. **Resolution:** Fixed 2026-09-28 on Frank's yes, docs only. The spec's outcomes now say `denied` ; the cipher signatures carry `boundTo` (the connection's `resourceId` , sealed in as GCM additional data) and "another person" joins the refusals; the ticket is used up by one DELETE on its fingerprint, the session's user and an unexpired `expiresAt` . Found on the way and fixed with it: the Google section still said the events scope "is checked before it is written"; it now names `calendar.events.owned` . The build log's Contracts row for the lock says the same. Waits for the next review to close. Closed 2026-09-30 by `/audit independent (scope: step 3.3)`: the spec's Data / contracts now say `denied` (outcomes list and Routes), the cipher signatures carry `boundTo` , the ticket is used up by one DELETE on fingerprint, session user and `expiresAt` , and the Google section names `calendar.events.owned` . All match the code read in this pass.

3/F-42 [P3] closed - Two names in the calendar folder read differently from their neighbours

File: backend/lib/calendar/use-oauth-ticket.ts:18 **Found:** 2026-09-28 by /audit independent (scope: step 3.2; lens: quality) **Why it matters:** The folder spells the same word two ways: `googleOAuthClient` beside `createOAuthTicket`, `useOAuthTicket`, `oauthStateFingerprint` and the `calendarOAuthState` table. And `useOAuthTicket` starts with `use`, which in this repo's frontend means a React hook; read from an import line it looks like one. Frank reviews names, and these are the ones 3.3 and 3.4 will import next to them. **Suggested fix:** One spelling across the folder (`OAuth`, matching the table whose name costs a migration to change), and a verb that says what happens, such as `redeemOAuthTicket` in `redeem-oauth-ticket.ts`. **Resolution:** Fixed 2026-09-29 on Frank's yes: `googleOAuthClient` is `googleOAuthClient`, matching the table and every other `OAuth` name; `useOAuthTicket` is `redeemOAuthTicket` in `redeem-oauth-ticket.ts` (its type `RedeemedOAuthTicketType`), so it no longer reads like a React hook. The spec follows. The code drawers on the build log keep the old names: they show the step as committed. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: step 3.3): no `googleOAuthClient`, `useOAuthTicket` or `use-oauth-ticket` is left in backend, frontend or shared source; the folder spells `OAuth` throughout, and step 3.3's new files import `googleOAuthClient` under that name.

3/F-43 [P3] closed - Two of the busy-times safeguards can be deleted and every test still passes

File: backend/lib/calendar/get-busy-times.ts:72 **Found:** 2026-09-30 by /audit independent (scope: step 3.3; lens: tests) **Why it matters:** The step promises two things the tests do not hold in place. First, when Google refuses a refresh at the same moment the owner reconnects, the fresh connection must not be marked "needs reconnecting". The code guards this (the mark only touches the row as it was read, then reads it again), but the only same-moment test uses a refresh Google accepts, so changing line 72 to match the row by id alone passes all 79 tests. The owner would reconnect and immediately be told to reconnect again. Second, a Google that hangs is cut off after ten seconds by `AbortSignal.timeout` in `google-calendar-provider.ts:36`; the "time-out" test makes the fake fetch throw at once, so deleting that line also passes, and a hung Google would then hold the booking check for minutes. Neither ever answers "free", so this is coverage, not a live bug. **Suggested fix:** Add one test where the token answer reconnects the person and then answers `invalid_grant`: the row must stay `connected` with the new keys, and free/busy must be asked with the reconnected access token. For the time-out, have the fake free/busy honour `init.signal` (wait for its abort) and run it under fake timers, or assert the request carries a signal. **Resolution:** Fixed 2026-09-30 on Frank's yes, tests only, no product code, in `backend/lib/calendar/get-busy-times.test.ts`. New test: the token answer reconnects the person and then answers `invalid_grant`; the row stays `connected` with the reconnected keys and free/busy is asked with the reconnected token. The old time-out test is replaced: the fake free/busy never answers and ends only when the request's own signal aborts (rejecting at once when there is none), and `AbortSignal.timeout` is replaced for the test by an already-expired signal, with a check that it was asked for 10000 ms. Proved: the mark matched by id alone fails the reconnect test; the signal line removed from `google-calendar-provider.ts` fails the time-out test; each file restored and compared. 46 shared and 80 backend tests pass. Waits for the next review to close. Closed 2026-09-30 by /audit

independent (scope: step 3.4, re-examining fb5ee1): `get-busy-times.test.ts` now has the reconnect-during-refusal test (the token answer reconnects, then answers `invalid_grant`; it asserts `connected`, the reconnected keys and the reconnected bearer), which fails if the mark at `get-busy-times.ts:67-71` matched by id alone because the call would throw; and the time-out test's fake free/busy only ends through the request's own signal, rejecting at once without one, with `AbortSignal.timeout` spied, checked for 10000 ms and restored in `finally`. Step 3.4 changed `get-busy-times.ts` only to look the plug up through `findCalendarProvider`, which leaves both guards as they were. Nothing new introduced; 93 backend tests pass.

3/F-44 [P3] closed - A connect that gives up still hands back a Gmail another connection uses

File: `backend/lib/calendar/finish-google-connect.ts:67` **Found:** 2026-09-30 by /audit independent (scope: step 3.4; lens: security) **Why it matters:** This step settled that Google keeps one permission per Gmail for our app, so handing any of its keys back switches off every connection that uses that Gmail, and it applied that rule in Disconnect and when switching accounts. The three places where a connect gives up still hand the new key back without asking: a permission box unticked (line 67), no refresh token or an unreadable sign-in token (line 73), and a failed save (line 110). So an owner of two businesses whose calendar is connected in the first, who connects the same Gmail in the second and unticks a box, silently switches off the first business's calendar. That card keeps saying "Connected" until its next busy read fails and it asks for a reconnect. Rare today (one person, two businesses, the same Gmail), and it heals with Reconnect. **Suggested fix:** Before handing a new key back, skip it when another connection already uses the same Gmail, with the same check `hand-back-calendar-permission.ts:30-42` makes (read the identity first on the unticked-box path, since it is read after the scope check today). One route test: Ben connected with `shared@gmail.com`, Ana's consent with the same Gmail missing a permission, nothing revoked. **Resolution:** Fixed 2026-09-30 on Frank's yes, with his own connection included. Checking it found the sharper case: pressing Reconnect on your own calendar and unticking a box handed back the same Gmail and cancelled your own working connection. The check is now its own file, `backend/lib/calendar/is-calendar-account-in-use.ts`, used by `hand-back-calendar-permission.ts` (leaving out the person's own row, which is the one going away) and by `finish-google-connect.ts`'s give-up paths (counting every row, the person's own included). The sign-in token is read before the scope check, so the unticked-box path knows the Gmail; an unreadable sign-in token names no Gmail and is still handed back. Two route tests: a Gmail another business uses is kept, and your own working connection survives a failed reconnect. Proved: the check removed from the give-up path fails both; restored and compared. 95 backend tests pass. Waits for the next review to close. Re-examined 2026-09-30 by /audit independent (scope: current, feature 3's final review at d76d355), kept `fixed`: the original defect is gone (`finish-google-connect.ts:70-78` skips the hand-back while any connection uses the Gmail; route tests `calendar-routes.test.ts:507-532` pass). But the repair counts every row as "in use", including a connection Google has already stopped accepting, which opens F-47. It closes together with F-47's repair. Closed 2026-09-30 by /audit independent (scope: current, feature 3's final review at b35c7ec): re-read `finish-google-connect.ts:67-126` and `is-calendar-account-in-`

`use.ts` . Every give-up path (a box unticked, no refresh token or an unreadable sign-in token, a failed save) goes through `handBack` , which skips the hand-back while any working connection uses that Gmail, the person's own included; the sign-in token is read before the scope check. With F-47's repair "in use" now means a working connection only, which is the case this finding protects. Route tests `calendar-routes.test.ts:507-532` pass. Nothing new introduced.

3/F-45 [P3] closed - The disconnect and account-switch tests depend on Gmail addresses other tests leave in the database

File: `backend/routes/calendar-routes.test.ts:463` **Found:** 2026-09-30 by /audit independent (scope: step 3.4; lens: tests) **Why it matters:** The new "is this Gmail still used by another connection" check looks across every business in the database. The route tests expect `ana.owner@gmail.com` to be used by nobody else, but `get-busy-times.test.ts:59` saves a live connection with that exact address for its own person, and Vitest runs the two files side by side against the same database. If they overlap, or a run of either file was stopped before its clean-up, six tests (lines 463, 494, 509, 526, 534, 558) get "still used" instead of the hand-back they expect and fail for a reason that has nothing to do with the code. Not seen failing: nine full backend runs in this pass all passed, because the busy-times file usually finishes first. **Suggested fix:** Give every Gmail in both files the run's own `tag ( ana.owner-${tag}@gmail.com , shared-${tag}@gmail.com )` , as the login emails already do, so no test can meet another test's or an earlier run's rows. **Resolution:** Fixed 2026-09-30 on Frank's yes, tests only. Every made-up Gmail in `calendar-routes.test.ts` (30 uses: `ana.owner` , `Ana.Owner` , `ana.second` , `shared` , `coworker`) now comes from `gmail(name)` , which adds the run's tag; `get-busy-times.test.ts` tags its one. Proved: a leftover business with a connection for the old `ana.owner@gmail.com` was added to the local database; the route tests as they were before the fix failed 9 tests with it present, the fixed file passed all 38; the leftover and the temporary copy of the old file were removed after. 95 backend tests pass. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: current, feature 3's final review at d76d355): every made-up Gmail in `calendar-routes.test.ts` comes from `gmail(name)` (line 36, the run's `tag`), and `get-busy-times.test.ts:59` tags its one; no untagged `@gmail.com` is left in either file. The full backend run passed 96 of 96. Nothing new introduced.

3/F-46 [P3] closed - Disconnect accepts a request from any page that carries the login cookie

File: `backend/routes/calendar-routes.ts:80` **Found:** 2026-09-30 by /audit independent (scope: step 3.4; lens: security) **Why it matters:** Disconnect is the first dashboard route that destroys something, and like `/calendar/connect` it checks the login cookie but not where the request came from. CORS only stops another site from reading the answer, not from sending a plain form post. In production the cookie is `SameSite=None` (`auth-server.ts:195`), so if a browser sends it with a form posted from another page, that page could disconnect an owner's calendar and hand the Google permission back. The cookie is also `Partitioned` , which in current browsers limits it to pages on the dashboard's own site, so whether this is reachable depends on the production domains, which are not decided yet. Development uses `SameSite=Lax` and is not affected. Every later dashboard route that changes data (bookings, hours) will

copy this shape. **Suggested fix:** Decide at deploy time, before the first client: either confirm the dashboard's site holds nothing but the dashboard, or add Hono's built-in `csrf({ origin: appOrigin })` (ships with Hono, no new package) to the dashboard routes in `app.ts`, with one test that a form post from another origin is refused. **Resolution:** Fixed 2026-09-30 on Frank's yes, now rather than at deploy. `backend/middleware/dashboard-middleware/dashboard-csrf-middleware.ts` is Hono's built-in `csrf({ origin: appOrigin })`, mounted in `app.ts` on `/calendar/` *between the CORS and no-store middleware; every later dashboard route that changes data mounts it the same way. It checks only requests a plain form could send (JSON needs a preflight, which the CORS rule already limits to the dashboard); a refused one gets Hono's plain 403, not the refusal shape, since no dashboard code ever receives it.* `/api/auth/` keeps Better Auth's own origin check. Test: a form-style Disconnect from another origin, and one with no Origin, are 403 and nothing is deleted; the dashboard's own still works. Proved: the middleware unmounted in `app.ts` fails the test; restored and compared. By hand: from the running dashboard, a real browser POST to `/calendar/connect` still answered 200 with Google's address. Only this change to `app.ts` was committed; Frank's uncommitted formatting edits there stay as they were. 96 backend tests pass. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: current, feature 3's final review at d76d355): `app.ts:20` mounts `dashboardCsrfMiddleware` on `/calendar/*`. Checked against the installed Hono 4.13.8: it skips GET (so Google's return to `/calendar/callback` is untouched), treats a request with no content type as a form, and then needs `Sec-Fetch-Site: same-origin` or `Origin` equal to `APP_ORIGIN`, so the dashboard's bodiless POSTs from its own origin pass and a cross-site form or no-Origin post is 403. Route test `calendar-routes.test.ts:630-655` passes. Nothing new introduced.

3/F-47 [P3] closed - A Reconnect that gives up leaves a new Google permission nobody holds

File: `backend/lib/calendar/is-calendar-account-in-use.ts:20` **Found:** 2026-09-30 by /audit independent (scope: current; lens: security) **Why it matters:** When Google has stopped accepting a calendar, the card says "Needs reconnecting". If the owner presses Reconnect and the connect then gives up (a box unticked at Google, no long-lived token, or a failed save), the fresh permission Google just granted should be handed back, because nothing is saved. It is not: the "is this Gmail still in use?" check (`finish-google-connect.ts:70-78`) counts the owner's own dead connection as a use, so the new permission is kept. The owner's Google account then lists the app with calendar access that no connection in the app holds. If they give up and press Disconnect, the answer says "Google had already stopped accepting this connection", while Google still shows the app. Rare, harmless in itself (the tokens are thrown away), and it heals on the next successful Reconnect, but it is the same leftover-permission problem F-37 fixed for account switches. **Suggested fix:** Count only connections Google still accepts: add `status = 'connected'` to the query in `is-calendar-account-in-use.ts` (a `needs_reconnect` row's permission is already refused, so keeping a new one for it protects nothing). One route test: Ana's connection set to `needs_reconnect`, a Reconnect with the same Gmail and a box unticked, the new token handed back. **Resolution:** Fixed 2026-09-30 on Frank's yes, before the merge. `is-calendar-account-in-use.ts` now counts only connections with `status = 'connected'`: a `needs_reconnect` one holds a key the

provider already refused, so there is nothing to cut off. This applies to all three callers alike (Disconnect's and the account switch's shared-Gmail rule, and the give-up paths); working connections stay protected as before. Test: Ana's connection set to `needs_reconnect`, a Reconnect with the same Gmail and a box unticked, the new refresh token handed back. Proved: the status condition removed fails it; restored and compared. 97 backend tests pass. F-44 closes with this repair's review. Waits for the next review to close. Closed 2026-09-30 by /audit independent (scope: current, feature 3's final review at b35c7ec): `is-calendar-account-in-use.ts:28` adds `status = 'connected'`, so a `needs_reconnect` row (its own included) no longer keeps a fresh permission alive on a give-up, and the new key is handed back. Checked against its three callers: a give-up still keeps a Gmail any working connection uses (F-44); Disconnect and the account switch still answer `still_used` for a working connection elsewhere, and a dead one elsewhere no longer blocks the hand-back, which cuts off nothing Google had not already refused; Disconnect's `already_stopped` is unchanged. Route test `calendar-routes.test.ts:533-553` (a dead connection, a Reconnect with the same Gmail and a box unticked, the new refresh token handed back) passes, and 97 of 97 backend tests pass. Nothing new introduced.

Independent review

Status: passed **Target commit:** b35c7ec2f41af6f4c37193b579547a15a864c855 **Base commit:** 5647cd4b38928a37993f9142ece705bd2af08f96 **Base ref:** main **Spec hash:** 657a3bea172bb2804d6e56a51190b3577d5b5b815c8cd94b68c0309dd4cf3080 **Prepared by:** claude **Builder model:** claude-opus-5-5 **Requested reviewer:** claude **Requested model:** claude-opus-5-5 **Requested execution:** automatic **Requested at:** 2026-09-30T21:36:43Z **Workflow:** regular **Check required:** no **Reviewer adapter:** claude **Reviewer model:** claude-opus-5-5 **Reviewer context:** fresh subagent **Actual execution:** automatic **Reviewed at:** 2026-09-30T21:41:38Z **Scope:** current **Lenses:** quality, security, performance, tests **Verdict:** passed **Check result:** not-required

Handoff

Review the active spec and the complete

`5647cd4b38928a37993f9142ece705bd2af08f96..b35c7ec2f41af6f4c37193b579547a15a864c855` delta in a fresh session or isolated subagent without the builder conversation. Run all Audit lenses from scratch. Run Check when required above. Do not edit product code, accept findings, or reuse the existing findings as the review scope.

Commands

- `git rev-parse HEAD`, `git merge-base main HEAD`, SHA-256 of `blueprint/context/current-feature.md`, `git status --porcelain --untracked-files=all` : pass (HEAD, merge base and spec hash match the request; only `blueprint/context/review.md` differs)
- `npm run test --workspace=@scheduleads-app/shared` : pass (5 files, 48 tests)

- `npm run test --workspace=backend` : pass (6 files, 97 tests, against the local seeded `scheduleads_dev` , Google faked)
- `npm run build --workspace=backend` : pass
- `npm run build --workspace=frontend` : pass
- `npm run lint --workspace=frontend` : pass
- `npm run format:check` : pass

Evidence

- Whole delta `5647cd4..b35c7ec` read (67 files): `backend/lib/calendar/*` , `backend/routes/calendar-routes.ts` (+ test), `backend/app.ts` , `server.ts` , `dashboard-csrf-middleware.ts` , `refuse.ts` , `auth-server.ts` , `backend/scripts/calendar-check.ts` and `find-calendar-check-target.ts` (+ test), `packages/shared` cipher, tables, migration 0004, outcomes list, dev-database guard and seed, `frontend` card, `api-client.ts` and `page.tsx` , `.env.example` , and the docs and skills changes.
- Security: the business and person come only from the session on every route; the OAuth ticket is stored as a SHA-256 fingerprint, bound to the session's login, unexpired and used up in one DELETE, so a forged or replayed callback ends in `expired` ; PKCE S256; the redirect target is fixed to `APP_ORIGIN` ; both calendar scopes are required; the `id_token`'s issuer, audience and `email_verified` are checked; tokens are sealed with AES-256-GCM bound to the person's id and never returned or logged (a database error logs only its Postgres code; Google's error words are limited to `[a-z_]+`); `/calendar/*` carries CORS, Hono's CSRF origin check and no-store.
- Busy times never answer "free" on failure: provider errors, time-outs, per-calendar errors, unreadable keys and a `needs_reconnect` row all throw; writes after a refresh touch only the row as read.
- F-44 and F-47 re-examined against `finish-google-connect.ts:67-126` and `is-calendar-account-in-use.ts:20-33` and all three callers of the in-use check; both repairs hold and introduce nothing new (route tests at `calendar-routes.test.ts:507-553`).
- Tests: each route and busy-times test builds its own throwaway business, login, person and tagged Gmail, clears them after, and refuses any database that is not local `*_dev` ; no `.only` , `.skip` or placeholder test found.

Findings

- None new. F-44 and F-47 closed. Earlier open or unverified entries (F-12, F-13, F-14, F-16, F-32, F-34), all P2 or P3 and outside this delta, left as they were.

Remaining risk

- Check was not required and not run; the real Google consent, busy block and third-party-apps page were proved by hand by the builder, not by this review.
- Two connects of the same person racing each other with different Gmails could leave one Google permission with no row (the previous account is read before the save, outside a transaction). Unverified and rare; not recorded as a finding.
- Production cookie and CSRF behaviour depends on the dashboard and API sharing one site (`app.` and `api.` of one domain), as `auth-server.ts` assumes; not provable locally.